第二部分

结构化并发应用程序

第6章

任务执行

大多数并发应用程序都是围绕“任务执行（Task Execution）”来构造的：任务通常是一些抽象的且离散的工作单元。通过把应用程序的工作分解到多个任务中，可以简化程序的组织结构，提供一种自然的事务边界来优化错误恢复过程，以及提供一种自然的并行工作结构来提升并发性。

6.1 在线程中执行任务

当围绕“任务执行”来设计应用程序结构时，第一步就是要找出清晰的任务边界。在理想情况下，各个任务之间是相互独立的：任务并不依赖于其他任务的状态、结果或边界效应。独立性有助于实现并发，因为如果存在足够多的处理资源，那么这些独立的任务都可以并行执行。为了在调度与负载均衡等过程中实现更高的灵活性，每项任务还应该表示应用程序的一小部分处理能力。

在正常的负载下，服务器应用程序应该同时表现出良好的吞吐量和快速的响应性。应用程序提供商希望程序支持尽可能多的用户，从而降低每个用户的服务成本，而用户则希望获得尽快的响应。而且，当负荷过载时，应用程序的性能应该是逐渐降低，而不是直接失败。要实现上述目标，应该选择清晰的任务边界以及明确的任务执行策略（请参见 6.2.2 节）。

大多数服务器应用程序都提供了一种自然的任务边界选择方式：以独立的客户请求为边界。Web 服务器、邮件服务器、文件服务器、EJB 容器以及数据库服务器等，这些服务器都能通过网络接受远程客户的连接请求。将独立的请求作为任务边界，既可以实现任务的独立性，又可